Fundamental Programming Structures in JAVA

OBJECTS AND CLASSES

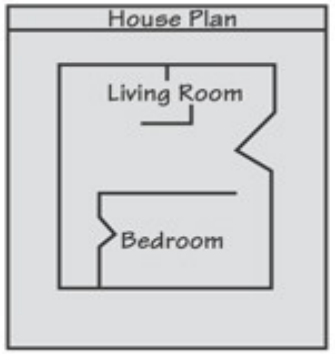
Objectives

- To describe objects and classes and use classes to model objects.
- To demonstrate how to define classes and create objects.
- To create objects using constructors.
- To access objects via object reference variables.
- To define a reference variable using a reference type.
- To access an object's data and methods using the object member access operator (.).
- To distinguish between object reference variables and primitive data type variables.
- To distinguish between instance and static variables and methods.

Object-Oriented Programming Terminologies

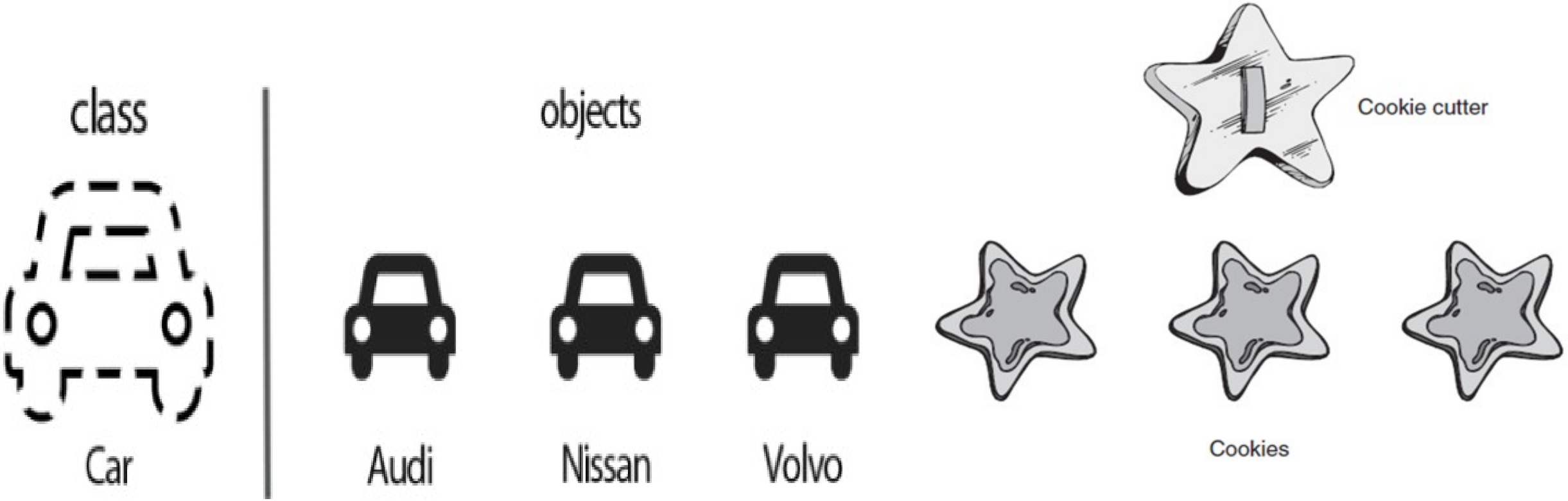
Classes and Objects

Blueprint that describes a house.



Instances of the house described by the blueprint.

Classes and Objects

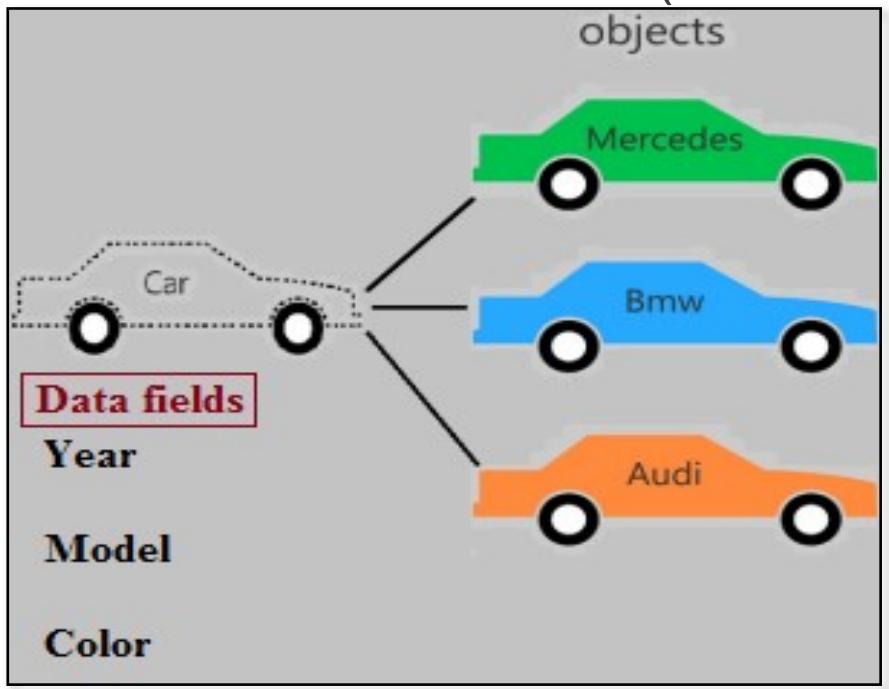


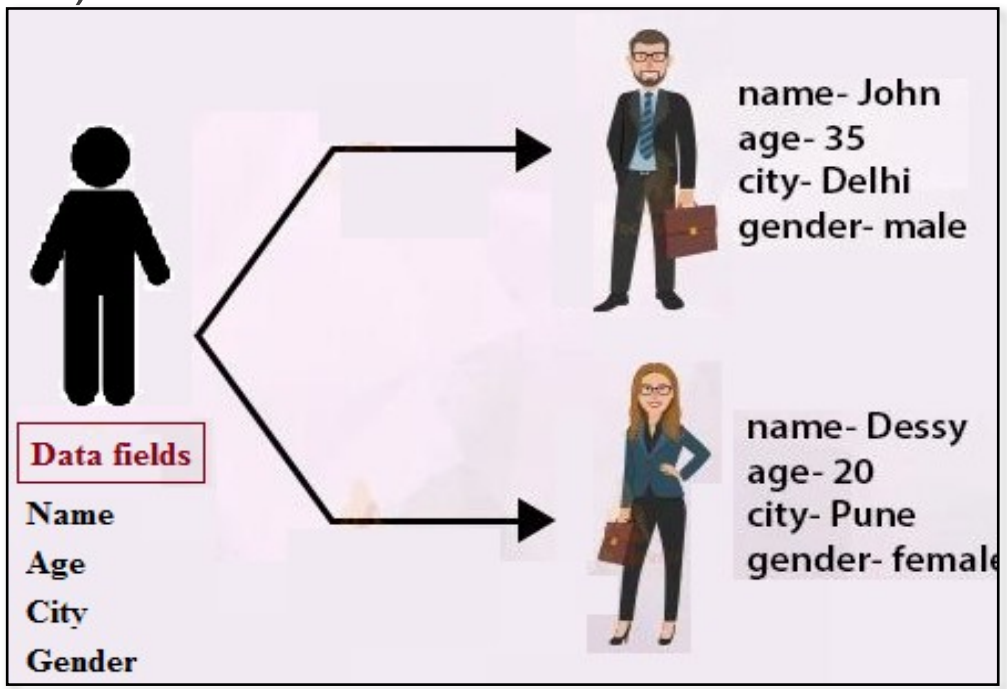
Classes and Objects

- **Class**: code that specifies the data attributes and methods of a particular type of object
  - Similar to a blueprint of a house or a cookie cutter.
- **Object** : an instance created from a class
  - Similar to a specific house built according to the blueprint or a specific cookie.
  - There can be many instances of one class.

Classes and Objects

- Classes are **user-defined** (programmer-defined) types.
  - **attributes** (data members)
  - **methods or behaviors**: (member methods)





Classes in Java

- Classes are constructs that define objects of the same type. A Java class uses variables to define data fields and methods to define behaviors. Additionally, a class provides a special type of methods, known as constructors, which are invoked to construct objects from the class.
- **Class : is a group of objects that share common properties and behavior.**
- For example, we can consider a car as a class that has characteristics like steering wheels, seats, brakes, etc. And its behavior is mobility. But we can say Honda City has a reg. number 4654 is an ‘object’ that belongs to the class ‘car’. It was a brief description of objects and classes. **Now we will understand the Java class in detail.**

Class Diagram

A class diagram

Class Name
Attribute$_1$ name: type Attribute$_2$ name: type . . . Attribute$_n$ name: type
List of methods

Each class has both a state (**data fields**) and behavior (**Methods**) :

Data fields defines the object attributes.

Methods define what the object does.

(we will explain it later)

Class Template

- Example 1: Design the class (Circle) that has (radius), (color) as its attributes:

Design Phase

Circle
radius: double Color: String

Class name

This class should be saved as (Circle.java).

Implementation (programming) Phase using java

Circle.java

```
1
2   public class Circle {// the begining of Circle class
3     //Attributes or Data Fields
4       double radius;
5       String color;
6   }//End of Circle class
7
```

A class allows the programmer to define all the properties that internally define the state of an object..

Class Template

- Example 2: Design the class (Rectangle) that has (length, width) as its attributes.

Design Phase

Rectangle
length: double width: double

Implementation (programming) Phase using java

Rectangle.java

```
1  public class Rectangle {
2      //Attributes or Data Fields
3       double length;
4       double width;
5  }
```

Class Syntax

- The general syntax for defining a class is:

```
modifier(s) class ClassIdentifier modifier(s)
{
    classMembers
}
```

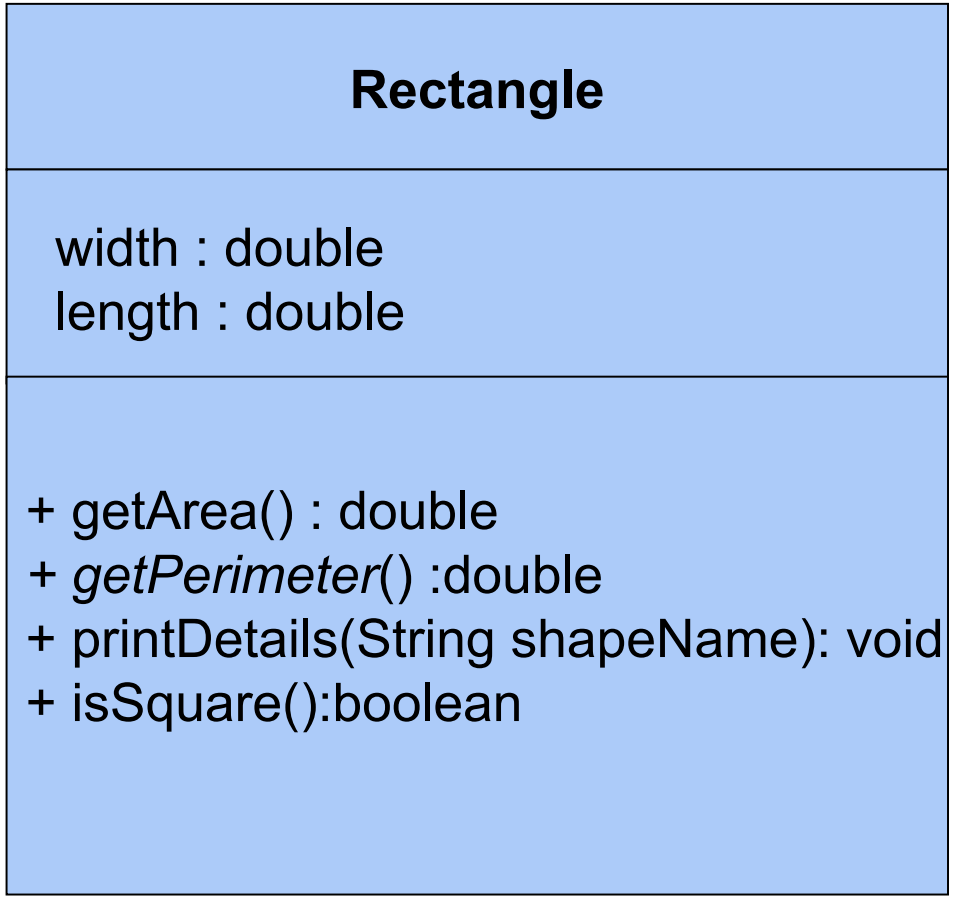
- Question: Design a class (Person) that has (name: String, age: int, length: double, salary: int, job: String, weight: double, and gender: char) as its attributes.

Writing a Class, (Step by Step)

Class name goes here

Attributes are listed here

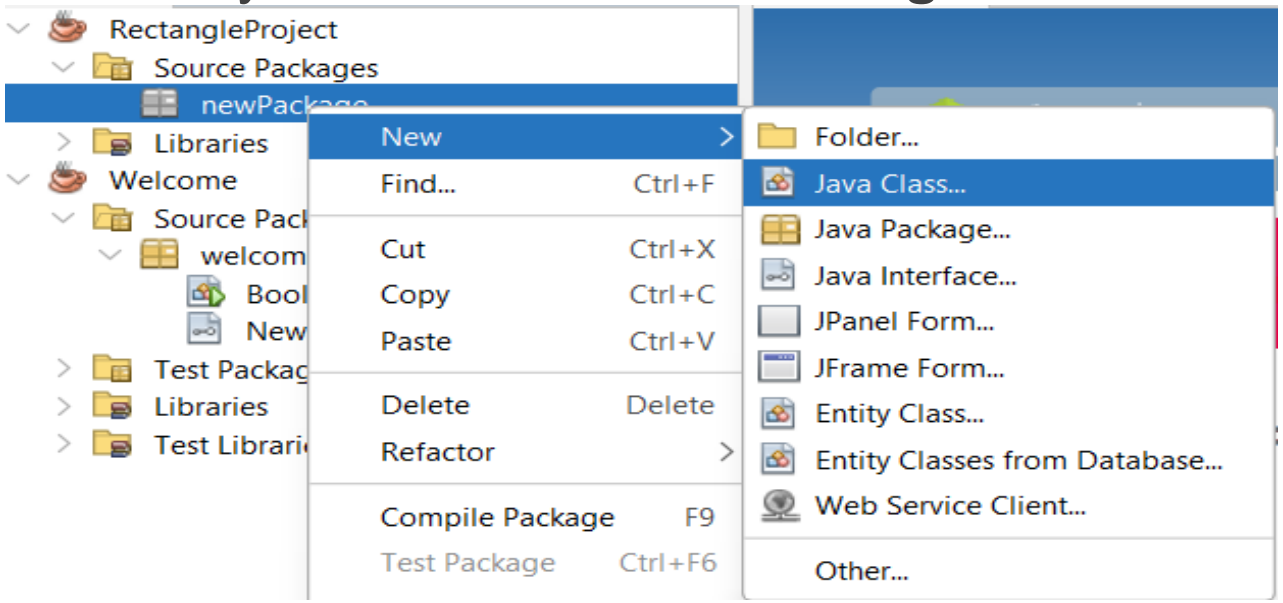
Methods are listed here

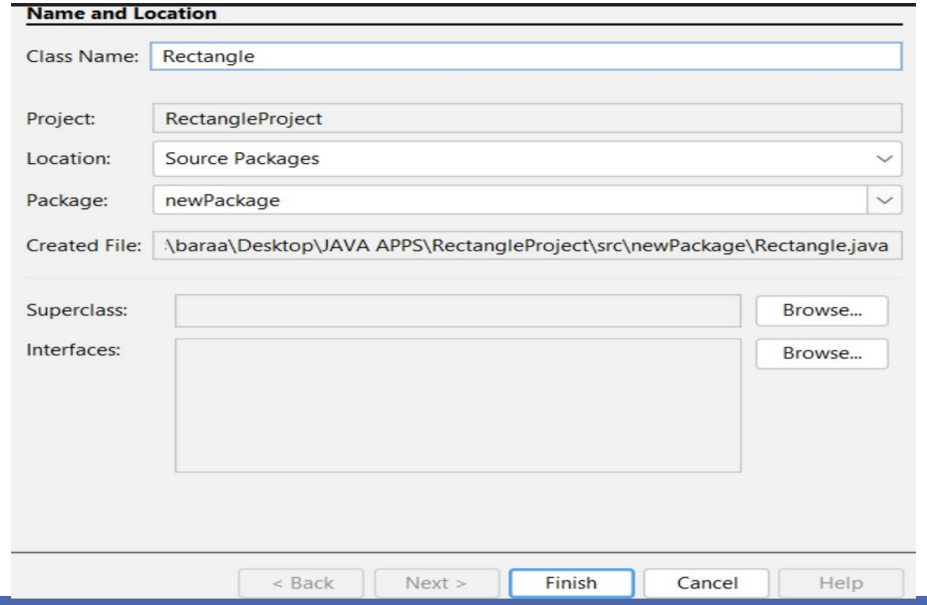


Unified Modeling Language (UML) provides a set of standard diagrams for graphically depicting object-oriented systems.

Cont. Writing a Class

- Create the **Rectangle** class:
  - Create a new project with an appropriate name.
  - Create a new package name it new package.
  - Finally create the class **Rectangle** in the newly created package.



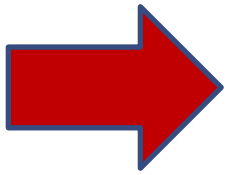


Cont. Writing a Class

- A **Rectangle** class will have the following data fields:
  - **length,** The length field will hold the rectangle's length.
  - **width,** The width field will hold the rectangle's width.

```
public class Rectangle
{

}
```



```
public class Rectangle
{
        double length;
        double width;
}
```

Instance Variables

- The data fields length and width in the rectangle class are known as **instance variables.**
- An Instance variable is tied to a specific instance of the class; it is not shared among objects of the same class

Class Name:
Rectangle
Data Field:
width
length
Methods:
getArea()
getPerimeter()

Rectangle object 1
Data fields:
width is 10
length is 7

Rectangle object 2
Data fields:
width is 15
Length is 10

Writing a Class, Step by Step

- The **Rectangle** class will also have the following methods:
  - **getArea()** will **return the area of the rectangle**, which is the result of the object's **length** multiplied by its **width**.
  - **getPerimeter()** will return the Perimeter of the rectangle, which is the result of the object's **2*length +2 *width.**

Writing a Class, Step by Step , cont..

- The **Rectangle** class will also have the following methods:
  - **printDetails(String shapeName)** will print the details of the rectangle such as length , width , area and shape name.
  - **isSquare()** will check wether the rectangle is a square or not

Instance Methods

- Method Declarations:
  - A method is a collection of statements that are grouped together to perform an operation.

Define a method

modifier

return value type

method name

Formal parameter

method header

```
public  void  printDetails(String shapeName)
{

  System.out.println("shape name "+shapeName);
  System.out.println("Length : "+length);
  System.out.println("width  : "+width);

}
```

Method signature

method body

```
x.printDetails("board");
```

Actual parameter

Writing a Class, Step by Step

Rectangle.java

Source History

```java
1   public class Rectangle {
2       double length;
3       double width;
4
5    public double getArea(){
6       double area=length*width;
7       return area;
8       }
9       public double getPerimeter()
10      {
11      return 2*length+2*width;
12      }
13
14     public void printDetails(String shapeName)
15      {
16      System.out.println("shape name:"+shapeName);
```

Data Fields

Instance Methods

Rectangle

Activate Windows
Go to Settings to activate Windows.

Writing a Class, Step by Step

Rectangle.java

Source History

```
13
14      public void printDetails(String shapeName)
15        {
16         System.out.println("shape name:"+shapeName);
17         System.out.println("length : "+length);
18         System.out.println("width : "+width);
19        }
20        public boolean isSquare(){
21
          if(length==width)
23            return true;
24        else
25            return false;
26        }
27    } // End
```

Instance Methods

Rectangle > printDetails >

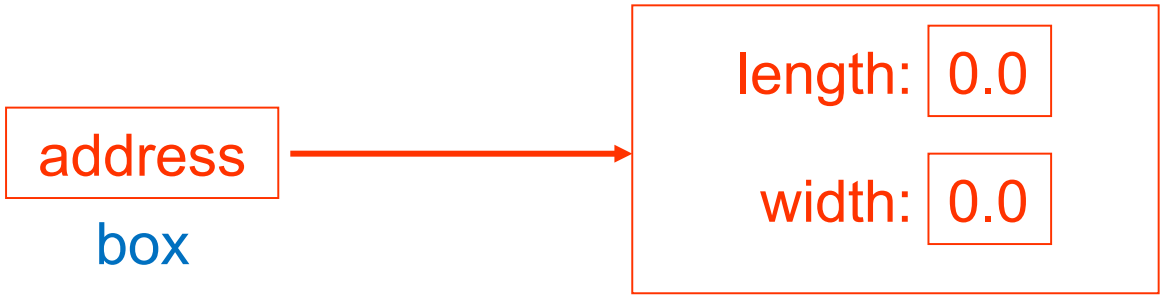
Activate Windows
Go to Settings to activate Windows.

Object Instantiation (Creating Object)

```
Rectangle box = new Rectangle ();
```

class name **box** reference variable **Rectangle** object

The **box** variable holds the address (Reference) of the Rectangle object.



Trace Method Invocation

Create an object

```
public static void main(String[] args) {
Rectangle box=new Rectangle();
box.width=10;
box.length=5;
System.out.print(box.getArea());

}
```

```
public double getArea(){

    double area=length*width;

    return area;

    }
```

Trace Method Invocation

width is now 10.0

```
public static void main(String[] args) {
Rectangle box=new Rectangle();
box.width=10;
box.length=5;
System.out.print(box.getArea());

}
```

```
public double getArea(){

    double area=length*width;

    return area;

    }
```

Trace Method Invocation

length is now 5.0

```
public static void main(String[] args) {
Rectangle box=new Rectangle();
box.width=10;
box.length=5;
System.out.print(box.getArea());

}
```

```
public double getArea(){

    double area=length*width;

    return area;

    }
```

Trace Method Invocation

Invoke getArea()

```
public static void main(String[] args) {
Rectangle box=new Rectangle();
box.width=10;
box.length=5;
System.out.print(box.getArea());

}
```

```
public double getArea(){

    double area=length*width;

    return area;

    }
```

Trace Method Invocation

Declare variable area

```
public static void main(String[] args) {
Rectangle box=new Rectangle();
box.width=10;
box.length=5;
System.out.print(box.getArea());

}
```

```
public double getArea(){

    double area=length*width;

    return area;

    }
```

Trace Method Invocation

Compute the area

```
public static void main(String[] args) {
Rectangle box=new Rectangle();
box.width=10;
box.length=5;
System.out.print(box.getArea());

}
```

```
public double getArea(){

    double area=length*width;

    return area;

    }
```

Trace Method Invocation

Compute the area and assign the result in area

```
public static void main(String[] args) {
Rectangle box=new Rectangle();
box.width=10;
box.length=5;
System.out.print(box.getArea());

}
```

```
public double getArea(){

    double area=length*width;

    return area;

    }
```

Trace Method Invocation

return the area, which is 50.0

```
public static void main(String[] args) {
Rectangle box=new Rectangle();
box.width=10;
box.length=5;
System.out.print(box.getArea());

}
```

```
public double getArea(){

    double area=length*width;

    return area;

    }
```

Constructors

- Classes can have special methods called constructors.
- A constructor is a method that is automatically called when an object is created.
- Constructors are used to perform operations at the time an object is created.
- Constructors typically initialize instance fields and perform other object initialization tasks.

Constructors

- Constructors have a few special properties that set them apart from normal methods.
  - Constructors have the same name as the class.
  - Constructors have no return type (not even void).
  - Constructors may not return any values.
  - Constructors are typically public.

The Default Constructor

- When an object is created, its constructor is always called.
- If you do not write a constructor, Java provides one when the class is compiled. The constructor that Java provides is known as the default constructor.
  - It sets all of the object's numeric fields to 0.
  - It sets all of the object's boolean fields to false.
  - It sets all of the object's reference variables to the special value null.
- The default constructor is a constructor with no parameters, used to initialize an object in a default configuration.
- The only time that Java provides a default constructor is when you do not write any constructor for a class.
- A default constructor is not provided by Java if a constructor is already written.

Writing Your Own No-Arg Constructor

- A constructor that does not accept arguments is known as a **no-arg constructor**.
- The default constructor (provided by Java) is a no-arg constructor.
- If you include even one constructor in your class, Java will not provide this default constructor
- If you include any constructors in your class, be sure to provide your own no-argument constructor as well
- We can write our own no-arg constructor

```
public Rectangle()
        {
            length = 1.0;
            width = 1.0;
        }
```

Parametrized Constructor

- Constructor like other methods can be overloaded.
- The constructor that takes parameters named a parametrized constructor.

```
public Rectangle(double len, double w)
   {  length = len;
      width = w;
     }
```

Parametrized Constructor

- What would happen when we execute the following calls?

```
Rectangle box1 = new Rectangle();
Rectangle box2 = new Rectangle(5.0, 10.0);
```

- The first call would use the no-arg constructor and box1 would have a **length** of **1.0** and **width** of **1.0**.
- The second call would use the original constructor and box2 would have a **length** of **5.0** and a **width** of **10.0**.

Summarization Constructors

- Constructors: are a special type of methods which have exactly **the same name as the class** and they are used to **construct objects of that class**.
  - You can write as many constructors as required in each class, each has different assignments (**Overloading**).
  - **Overloading**: is when two or more methods have the same name but different parameters within the same class.
  - Constructors are example of overloading. All the constructors of a specific class have the **same name** (the class name), but **the list of parameters should be different between each others**.

Constructors Types

Circle.java

```java
public class Circle {

    double radius;
    String color;
//Constructors:

    public Circle(double x, String y) {
        radius = x;
        color = y;
    }

    public Circle() {
    }

    public Circle(String x) {
        color = x;
    }

    public Circle(double y) {
        radius = y;
    }
}//End of class
```

Parameterized constructor

No argument constructor

One parameter constructor

Constructors Types

Circle.java | test.java

```java
public class test {

    public static void main(String[] args) {
    Circle r1=new Circle ();
    Circle r2=new Circle (5, "red");
    Circle r3=new Circle (12);
    Circle r4=new Circle ("green",14);
    System.out.println("Zero parameter constructor:"+ r1.color+ ","+ r1.radius);
    System.out.println("full parameter constructor:"+ r2.color+ ","+ r2.radius);
    System.out.println("one parameter constructor:" + r3.color+ ","+ r3.radius);

    }

}
```

Code and Error explanation in next slides

Constructors Types: Explanation

- Line 4: creates an object (r1) from the class circle using the constructor without parameters in line 13 in the circle class. Note that there no default constructor in the class now, since there is at least one user defined constructor written.
  - (r1) now has radius = 0.0 and color = null.
  - The code of this constructor can be left empty ({}) so the default values are going to be assigned to the attributes. Instead of this, you can assign any required values to the attributes.
- Line 5: creates an object (r2) from the class circle using the constructor with full parameters in line 8 in the circle class, passing the values 5 and "red" to be the values of radius and color attributes respectively.

Constructors Types: Explanation

- Line 6 : creates an object (r3) from the class circle using the constructor with one parameter in line 16 in the circle class, passing the values 12 to be the value of radius.
- The constructors in line (7) generates error, since there are no constructors in the class matching the given parameters.

Output - Circle (run)

```
run:
Zero parameter constructor:null,0.0
full parameter constructor:red,5.0
one parameter constructor:null,12.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Example: Circle Class with Constructors

```
class Circle {
  /** The radius of this circle */
  double radius = 1.0;

  /** Construct a circle object */
  Circle() {
  }

  /** Construct a circle object */
  Circle(double newRadius) {
    radius = newRadius;
  }

  /** Return the area of this circle */
  double getArea() {
    return radius * radius * 3.14159;
  }
}
```

Data field

Constructors

Method

Trace Code

Declare myCircle

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```

Memory

myCircle declared In the memory

`myCircle`	**no value**

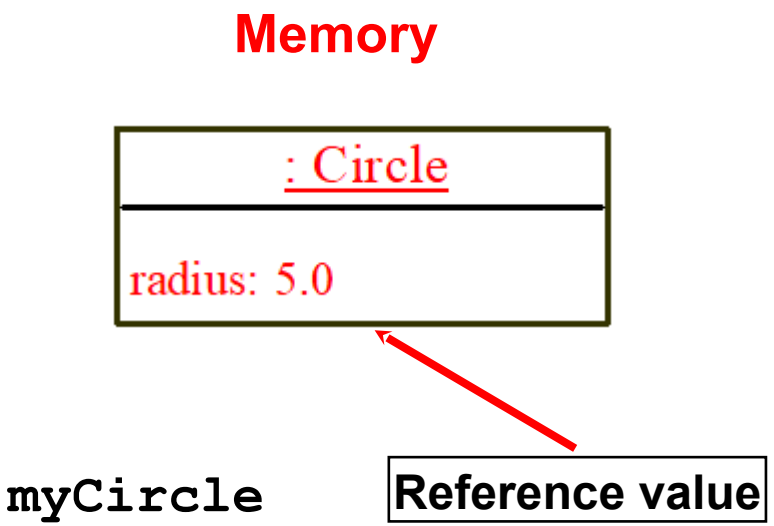
Trace Code

Assign object reference to myCircle

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```



Trace Code

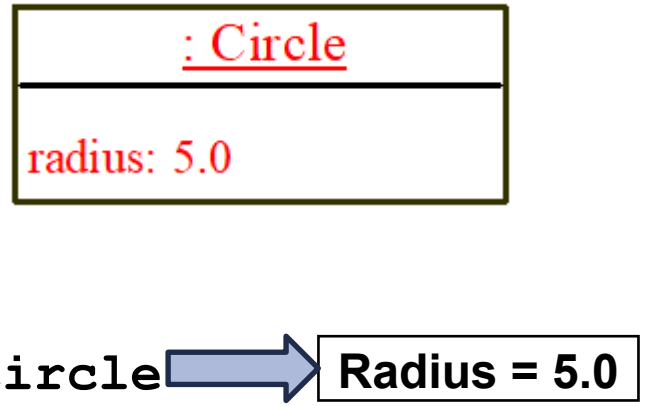
New circle created

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```

Memory



Trace Code

Declare yourCircle

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```

Memory

yourCircle declared
In the memory

`yourCircle`	**no value**

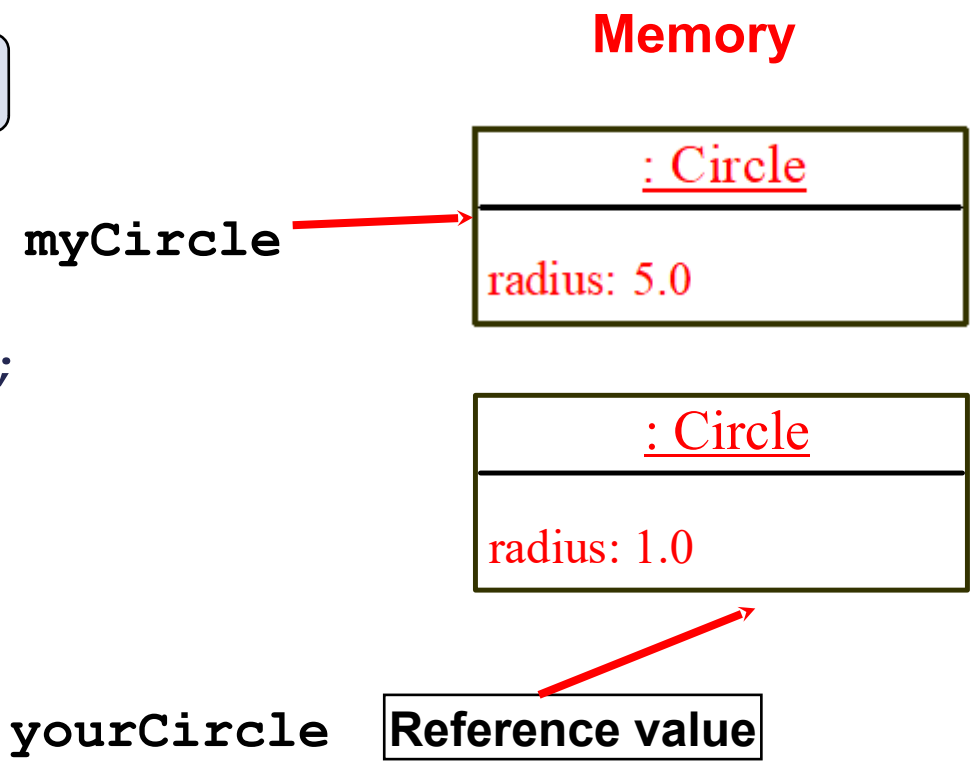
Trace Code

Assign object reference to yourCircle

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```



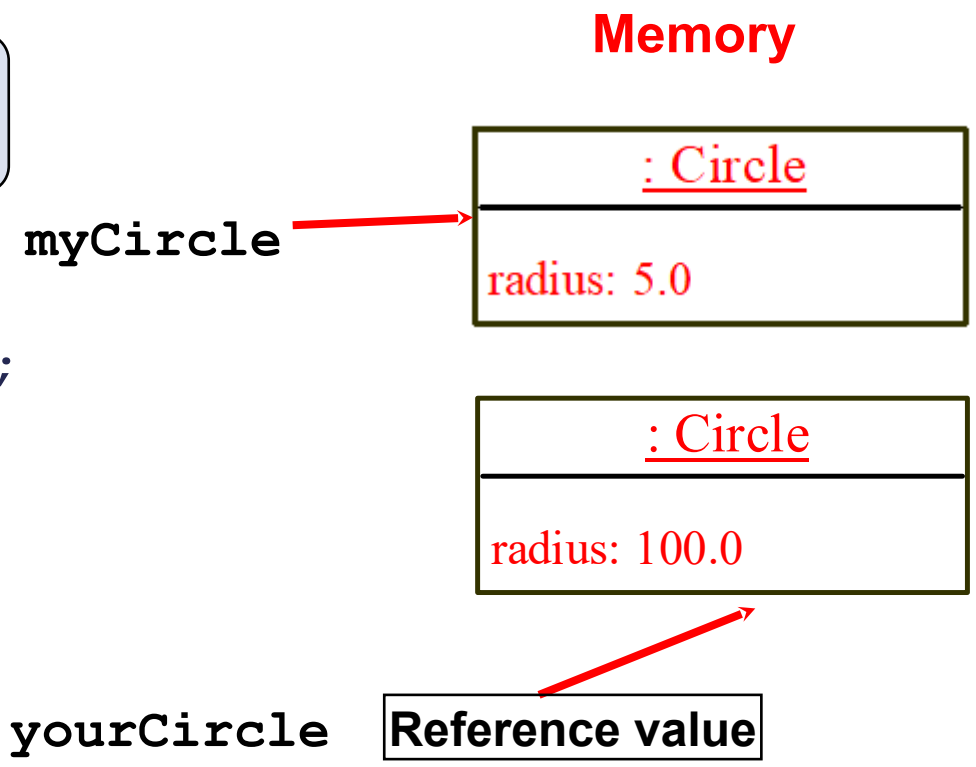
Trace Code

Change radius value in yourCircle

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```



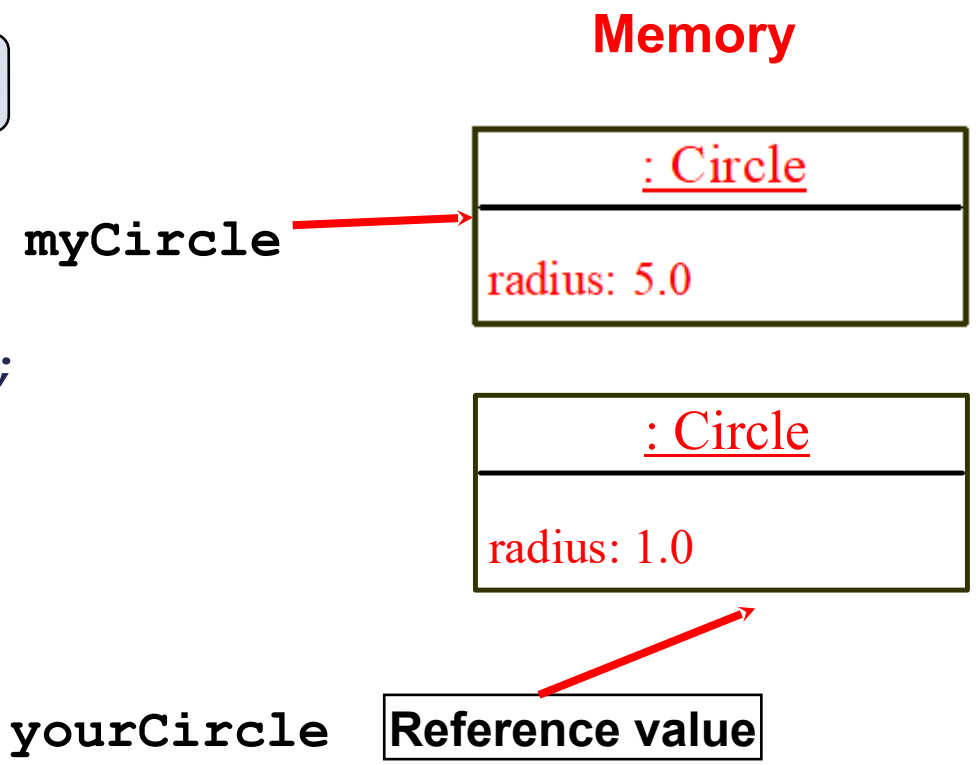
Trace Code

Assign object reference to yourCircle

```
Circle myCircle = new Circle(5.0);

Circle yourCircle = new Circle();

yourCircle.radius = 100;
```



Exercise

- Design a class named (Book) that has the following members:
  - Two instance variables number of type int and name of type String
  - Three different constructors.
  - display( ) method to print the details of the book .
- In the main class , create three objects of type (Book) using different constructors to represent three books and print the details for each book.

References

- [1] Introduction to Java Programming, Comprehensive Version, Student Value Edition, Daniel Liang, 12th Edition, Pearson 2019
- lecture Notes :edugate.asu.edu.jo.
- lecture Videos : YouTube channel.
- Lab manual ???